MAPLE Architecture Guide

Version: 1.0 — Spring 2026
Maintainers: Prof. Brian Gormanly, Dr. Bowu Zhang
Last Updated: February 2026

Purpose

This guide establishes shared technical conventions for all MAPLE modules developed in CMPT 394. Following these conventions ensures that:

- Modules can eventually be composed into a unified MAPLE platform.
- Code and documentation are consistent enough for handoff to future teams (including the Fall 2026 production assistant and potential capping teams).
- Faculty can evaluate projects against a common technical baseline.
- Teams working on related modules (especially M1 and M2) can coordinate effectively.

This is a living document. If you encounter a situation where these conventions don't make sense for your module, raise it with the faculty PIs — we'd rather update the guide than have teams silently diverge.

---

1. Technology Stack

Required

All teams must choose one of the two approved backend stacks and use it consistently throughout the project. Your choice should be justified in your design doc.

Layer	Technology	Notes
Backend (Option A)	Node.js with Express	Strong ecosystem for full-stack JavaScript teams. Well-documented, large community, straightforward deployment.

Layer	Technology	Notes
Backend (Option B)	Python with FastAPI	Natural fit for teams doing heavier data processing, ML integration, or working with Python-native AI/embedding libraries. Excellent async support and automatic API documentation.
Frontend	Angular	Other modern frameworks (React, Svelte) permitted with justification. Vanilla JS is not recommended for MAPLE modules due to component complexity.
Database	PostgreSQL	Primary relational store. Use for user data, application state, structured domain data.
Vector Store	pgvector (PostgreSQL extension) or Pinecone	pgvector is preferred for simplicity (single database). Pinecone is acceptable if your retrieval needs justify a dedicated vector service.
Version Control	Git / GitHub	All code in the MAPLE GitHub organization (details below).
Package Management	npm (Node) or pip with `requirements.txt` (Python); npm for frontend	Maintain clean dependency files. Pin major versions. Python teams should also consider using a virtual environment (`venv`).

Stack consistency rule: Once your team commits to a backend stack in the design doc, stick with it. Switching mid-project is costly and rarely justified. M1 and M2 must use the same backend stack as part of their coordination requirement.

Recommended

These are strong recommendations, not requirements. Justify alternatives in your design doc.

Concern	Recommendation	Notes
LLM API	Anthropic Claude (via API) or OpenAI GPT-4o	Choose based on your module's needs. Document the trade-offs in your design doc.
Embedding Model	OpenAI `text-embedding-3-small`	Good balance of quality, cost, and dimension size. M1 and M2 must use the same embedding model.
Cloud Hosting	DigitalOcean	Low cost, straightforward setup, good documentation. AWS and Azure are fine but add complexity.
ORM	Prisma or Drizzle (Node); SQLAlchemy (Python)	Helps keep database access consistent and type-safe.
Environment Management	dotenv + `.env` files	Never commit `.env` to the repository. Use `.env.example` with placeholder values.

Prohibited

- **No hardcoded API keys, secrets, or credentials anywhere in source code.** Use environment variables exclusively.
- **No `localStorage` or `sessionStorage` for sensitive data** in production deployments.
- **No direct database access from frontend code.** All data access goes through your backend API.

---

2. Repository Structure

All MAPLE modules should follow this repository structure. You don't need every directory from day one, but your final submission should conform to this layout.

maple-[module-id]/

```
├── README.md                  # Project overview, setup, architecture, AI approach
├── .env.example               # Environment variable template (no real values)
├── .gitignore
├── package.json               # Node teams (root or per-directory)
├── requirements.txt           # Python teams (or use per-directory)
│
├── docs/
│   ├── design-doc.md          # Project Design Doc (Week 8 deliverable)
│   ├── reconciliation.md      # Design Doc Reconciliation (Final Project deliverable)
│   ├── architecture-diagram.png # System architecture diagram
│   ├── api-spec.md            # API endpoint documentation
│   └── deployment.md          # Deployment process and infrastructure
│
├── prompts/
│   ├── system/                # Production system prompts used by your module
│   │   └── main-system-prompt.md
│   └── dev/                   # AI development log (per team member)
│       ├── alice/
│       │   └── [yyyyMMdd]-[prompt-goal].txt
│       └── bob/
│           └── [yyyyMMdd]-[prompt-goal].txt
│
```

```
├── server/                 # Backend application
│   ├── src/                # Node teams
│   │   ├── index.js        #  Entry point
│   │   ├── routes/         #  API route definitions
│   │   ├── controllers/    #  Request handling logic
│   │   ├── services/       #  Business logic and AI integration
│   │   ├── models/         #  Database models / ORM definitions
│   │   ├── middleware/     #  Auth, error handling, logging
│   │   └── utils/          #  Shared utilities
│   ├── app/                # Python teams
│   │   ├── main.py         #  Entry point
│   │   ├── routers/        #  API route definitions
│   │   ├── services/       #  Business logic and AI integration
│   │   ├── models/         #  Database models / ORM definitions
│   │   ├── middleware/     #  Auth, error handling, logging
│   │   └── utils/          #  Shared utilities
│   ├── tests/              # Backend tests
│   └── package.json        # Node teams
│   └── requirements.txt    # Python teams
│
├── client/                 # Frontend application
```

```
│   ├── src/
│   │   ├── components/        # UI components
│   │   ├── pages/             # Page-level views
│   │   ├── services/          # API client functions
│   │   └── utils/             # Frontend utilities
│   ├── public/
│   └── package.json
│
├── data/                      # Data pipeline
│   ├── raw/                   # Raw collected data (gitignored if large)
│   ├── processed/             # Cleaned/chunked data ready for ingestion
│   └── scripts/               # Data collection, processing, and embedding scripts
│
└── eval/                      # Evaluation
    ├── test-cases/            # Test inputs and expected outputs
    ├── results/               # Evaluation run results
    └── scripts/               # Evaluation automation scripts
```

> **Note:** Use the directory structure that matches your chosen stack. You don't need both `src/` and `app/` — pick the one that applies. The key requirement is that the overall layout (docs, prompts, server, client, data, eval) stays consistent across all modules.

Repository Naming

Repositories should be named: `maple-[module-id]`

Examples: `maple-m1`, `maple-m2`, `maple-m3`, `maple-m4`, `maple-a1`, `maple-a2`, `maple-a3`, `maple-a4`, `maple-a5`

Branching Strategy

- **`main`** — Always deployable. Protected branch; merge via pull request only.
- **`dev`** — Active development branch. Merge to `main` when stable.
- **Feature branches** — Named `feature/[short-description]` (e.g., `feature/embedding-pipeline`, `feature/rubric-input-form`). Branch from and merge back to `dev`.

Every team member should have commits on the repository. Commit history is part of the evaluation and will be analyzed by the A4 team's Repository Analyzer (yes, your repos may be used as test data — write history you'd be proud of).

---

3. API Design Conventions

All MAPLE modules expose REST APIs. Following shared conventions here is critical for future platform integration.

Base URL Pattern

https://[your-domain]/api/v1/[module-prefix]/[resource]

Module prefixes:

Module	Prefix
M1: Course Catalog	`/catalog`
M2: Degree Planning	`/advising`
M3: Campus Services	`/campus`
M4: Career Finder	`/career`
A1: Code Evaluator	`/code-eval`
A2: Written Assessor	`/written-eval`
A3: Doc Reviewer	`/doc-eval`
A4: Repo Analyzer	`/repo-eval`

Module	Prefix
A5: Rubric Engine	`/rubric`

Standard Response Format

All API responses must use this envelope structure:


```
{
  "success": true,
  "data": { },
  "error": null,
  "metadata": {
    "timestamp": "2026-04-15T14:30:00Z",
    "module": "m1",
    "version": "1.0.0"
  }
}
```


Error responses:


```
{
  "success": false,
  "data": null,
  "error": {
    "code": "RETRIEVAL_FAILED",
    "message": "Unable to find relevant information for your query.",
    "details": "No chunks exceeded the similarity threshold of 0.75."
```

```
  },
  "metadata": {
    "timestamp": "2026-04-15T14:30:00Z",
    "module": "m1",
    "version": "1.0.0"
  }
}
```

Standard Error Codes

Use these error codes consistently across modules:

Code	HTTP Status	Meaning
`VALIDATION_ERROR`	400	Request input failed validation
`UNAUTHORIZED`	401	Missing or invalid authentication
`FORBIDDEN`	403	Authenticated but not permitted
`NOT_FOUND`	404	Resource not found
`RETRIEVAL_FAILED`	422	AI retrieval returned no usable results
`AI_ERROR`	502	LLM API call failed or returned unusable output
`RATE_LIMITED`	429	Too many requests
`INTERNAL_ERROR`	500	Unexpected server error

MaristChat Conversation Endpoint

All MaristChat modules (M1–M4) must implement this endpoint:

POST /api/v1/[prefix]/chat

Request:


```
{
  "message": "What CS electives are available on Tuesdays?",
  "conversation_id": "conv_abc123",
  "context": {
    "user_major": "Computer Science",
    "user_year": "Junior"
  }
}
```


Response:


```
{
  "success": true,
  "data": {
    "response": "Based on the Spring 2026 schedule, the following CS electives meet on Tuesdays...",
    "conversation_id": "conv_abc123",
    "sources": [
      {
        "title": "Spring 2026 Course Schedule — CMPT",
        "chunk_id": "chunk_xyz789",
        "relevance_score": 0.89
```

```
      }
    ],
    "confidence": "high"
  },
  "error": null,
  "metadata": {
    "timestamp": "2026-04-15T14:30:00Z",
    "module": "m1",
    "version": "1.0.0",
    "model": "claude-sonnet-4-20250514",
    "latency_ms": 1230
  }
}
```


The `sources` array is required for all MaristChat responses — it enables source attribution in the UI and is essential for evaluating retrieval quality.

The `confidence` field (`"high"`, `"medium"`, `"low"`, `"none"`) indicates how well-supported the response is by retrieved sources. This allows the UI to display appropriate caveats.

The `context` field in the request is optional but should be supported. It allows the future unified platform to pass user context across modules.

Autograding Submission Endpoint

All Autograding modules (A1–A5) must implement this endpoint:

POST /api/v1/[prefix]/evaluate

Request (A2 example — Written Work):

{
  "submission_id": "sub_def456",
  "submission_type": "essay",
  "content": "The essay text or a reference to the uploaded file...",
  "rubric_id": "rubric_ghi789",
  "options": {
    "provide_feedback": true,
    "feedback_detail": "detailed"
  }
}

Response:

{
  "success": true,
  "data": {
    "submission_id": "sub_def456",
    "overall_score": 82,
    "max_score": 100,
    "criteria_scores": [
      {
        "criterion": "Thesis & Argument",
        "score": 18,

"max_score": 25,

"level": "Proficient",

"feedback": "Your thesis is clearly stated and arguable. Consider strengthening the connection between your thesis and the evidence in paragraphs 3 and 4."

},

{

"criterion": "Evidence & Support",

"score": 20,

"max_score": 25,

"level": "Proficient",

"feedback": "Good use of primary sources. The analysis in section 2 could go deeper — you describe the evidence but don't fully explain how it supports your argument."

}

],

"overall_feedback": "This is a well-structured essay with a clear argument...",

"flags": []

},

"error": null,

"metadata": {

"timestamp": "2026-04-15T14:30:00Z",

"module": "a2",

"version": "1.0.0",

"model": "claude-sonnet-4-20250514",

```
    "latency_ms": 4500,

    "evaluation_passes": 2

  }

}
```

The `criteria_scores` array maps directly to rubric criteria. Each entry must include the criterion name, numeric score, max score, performance level label, and criterion-specific feedback.

The `flags` array is for cases that need human attention (e.g., `"potential_plagiarism"`, `"off_topic"`, `"insufficient_length"`, `"ai_confidence_low"`).

The `evaluation_passes` field in metadata records how many LLM evaluation passes were used, supporting the multi-pass assessment strategies discussed in the A2 project description.

---

4. AI Integration Conventions

System Prompt Management

Store all production system prompts as version-controlled files in `prompts/system/`. This is not optional — system prompts are code, and they should be tracked with the same discipline as any other source file.

System prompts should include:

- **Role definition:** What persona is the AI adopting?
- **Scope boundaries:** What topics is it allowed to address? What should it refuse or redirect?
- **Output format instructions:** How should responses be structured?
- **Source attribution instructions:** How should the AI reference retrieved information?
- **Safety guardrails:** What should the AI do when it doesn't know, when it encounters inappropriate input, or when retrieval fails?

LLM API Call Wrapper

Every module should route LLM API calls through a shared service layer (not scattered API calls throughout the codebase). This service should:

- Log every call: prompt, response, model, token usage, latency.
- Handle retries with exponential backoff for transient failures.
- Enforce a timeout (recommend 30 seconds for standard calls, 60 seconds for complex evaluations).
- Catch and normalize errors from the LLM provider.
- Track cost (input tokens × rate + output tokens × rate) per call.

Example service interface (Node.js):

```
// services/llm.js

const llm = {

  /**

   * Send a completion request to the configured LLM provider.

   * @param {Object} options

   * @param {string} options.systemPrompt - System prompt text

   * @param {Array}  options.messages - Conversation messages [{role, content}]

   * @param {string} options.model - Model identifier

   * @param {number} options.maxTokens - Maximum response tokens

   * @param {number} options.temperature - Sampling temperature (0-1)

   * @returns {Object} { content, usage: { inputTokens, outputTokens, cost }, latencyMs }

   */

  async complete({ systemPrompt, messages, model, maxTokens, temperature }) {

    // Implementation with logging, retries, error handling

  }

};
```

Example service interface (Python / FastAPI):

```
# services/llm.py

from dataclasses import dataclass

@dataclass

class LLMUsage:

    input_tokens: int

    output_tokens: int

    cost_usd: float

@dataclass

class LLMResponse:

    content: str

    usage: LLMUsage

    latency_ms: int

async def complete(

    system_prompt: str,

    messages: list[dict],

    model: str,

    max_tokens: int = 1024,

    temperature: float = 0.7,

) -> LLMResponse:

    """

    Send a completion request to the configured LLM provider.
```

```
    Handles logging, retries with backoff, timeout, and error normalization.

    """

    # Implementation with logging, retries, error handling

    ...
```

Whichever stack you use, the contract is the same: a single entry point for all LLM calls, with structured logging and error handling built in.

Retrieval Conventions (MaristChat Modules)

MaristChat modules using RAG should standardize on:

- **Chunk metadata:** Every stored chunk must include `source_title`, `source_url` (if applicable), `source_type`, `last_updated`, and `chunk_index`. This metadata powers source attribution in responses.
- **Similarity threshold:** Do not return chunks below a relevance score of 0.70. If no chunks meet this threshold, the module should acknowledge uncertainty rather than hallucinate. Adjust this threshold based on your evaluation results, but document your choice.
- **Top-k retrieval:** Retrieve the top 5 chunks by default. Your design doc should justify a different number if you use one.
- **Context window budgeting:** Be intentional about how much of the context window you allocate to retrieved chunks vs. system prompt vs. conversation history. Document your allocation in the design doc.

Evaluation Data Conventions (Autograding Modules)

Autograding modules should standardize on:

- **Rubric format:** All rubrics should be stored as structured JSON conforming to this schema:

```
{

  "rubric_id": "rubric_ghi789",

  "title": "Argumentative Essay Rubric",

  "total_points": 100,

  "criteria": [
```

```
    {
      "name": "Thesis & Argument",
      "max_points": 25,
      "levels": [
        { "label": "Exemplary",    "points": 25, "description": "..." },
        { "label": "Proficient",   "points": 18, "description": "..." },
        { "label": "Developing",   "points": 12, "description": "..." },
        { "label": "Beginning",    "points": 5,  "description": "..." }
      ]
    }
  ]
}
```

This shared rubric format is especially important for the A5 team (Rubric Engine), whose output should conform to this schema so that A1, A2, and A3 can consume rubrics generated by A5.

---

5. Logging & Observability

Every module must implement structured logging for AI interactions. This is not just good practice — the MAPLE pilot evaluation depends on usage data.

Required Log Fields

Every AI interaction (LLM call, retrieval query, evaluation pass) should log:

```
{
  "timestamp": "2026-04-15T14:30:00.123Z",
```

```
  "module": "m1",
  "event_type": "llm_call",
  "conversation_id": "conv_abc123",
  "model": "claude-sonnet-4-20250514",
  "input_tokens": 1250,
  "output_tokens": 340,
  "latency_ms": 1230,
  "estimated_cost_usd": 0.0067,
  "success": true,
  "error": null
}
```


For retrieval operations, also log:


```
{
  "event_type": "retrieval",
  "query": "CS electives on Tuesdays",
  "chunks_retrieved": 5,
  "top_score": 0.89,
  "min_score": 0.72,
  "threshold_applied": 0.70
}
```

Where to Log

For the prototype and pilot phase, logging to structured JSON files is sufficient (one file per day, rotated). If you want to use a logging service (e.g., Winston with file transport, or a simple custom logger), that's fine — just ensure the output is structured JSON, not free-text.

---

6. Security Baseline

These are minimum security requirements for all modules:

- **No secrets in code.** API keys, database credentials, and any sensitive configuration must live in environment variables loaded from `.env` files that are `.gitignore`-d.
- **Input validation.** All API endpoints must validate input before processing. Don't pass raw user input directly to database queries or LLM prompts without sanitization.
- **CORS configuration.** Set appropriate CORS headers — don't use `Access-Control-Allow-Origin: *` in production.
- **Rate limiting.** Implement basic rate limiting on AI-powered endpoints to prevent accidental cost spikes. A simple per-IP limit (e.g., 30 requests per minute) is sufficient for the pilot.
- **HTTPS only.** Production deployments must use TLS. No exceptions.
- **Sandboxed execution (A1 only).** The Code Submission Evaluator must never execute student code in the same environment as the application server. Use Docker containers or a similar isolation mechanism.

---

7. Documentation Standards

README.md

Every module's README must include:

1. **Project title and MAPLE module identifier** (e.g., "M1: Course Catalog & Registration Assistant")
2. **One-paragraph description** of what the module does
3. **Architecture overview** with a link to the full architecture diagram
4. **Tech stack** — languages, frameworks, databases, AI services, cloud services
5. **Setup instructions** — step-by-step guide to run the application locally, including:
    - Prerequisites (Node version, database setup, etc.)
    - Environment variable configuration (reference `.env.example`)

    - Database migration/seeding commands
    - Data pipeline initialization (how to populate the vector store or load test data)
    - Start commands for backend and frontend
6. **API documentation** — link to `docs/api-spec.md` or inline summary of key endpoints
7. **AI integration summary** — which model, what approach (RAG, function calling, etc.), key design decisions
8. **Deployment** — URL to live deployment, deployment process summary
9. **Evaluation results** — summary of how the AI component performs
10. **Team members** — names and primary responsibilities
11. **AI disclosure** — brief note on how AI tools were used in development

Code Comments

Comment your code where the *why* isn't obvious from the *what*. Specifically:

- Every system prompt should have comments explaining design decisions.
- AI service integration code should explain the expected input/output contract.
- Data pipeline transformations should explain why data is being shaped a particular way.
- Don't comment obvious code (`// increment counter` above `counter++`).

---

8. M1/M2 Coordination Protocol

Teams M1 (Course Catalog) and M2 (Degree Planning) have a formal coordination requirement due to their overlapping academic data domains. In addition to the conventions in this guide, M1 and M2 must:

- **Use the same embedding model** for vector search (coordinate on the choice and document it jointly).
- **Use the same LLM provider and model** for their primary chat endpoint.
- **Align data ingestion pipelines** — course catalog data (M1) and degree requirement data (M2) should be processed with compatible chunking strategies and consistent metadata schemas. Prerequisite information, in particular, must be represented consistently since both modules need it.
- **Hold weekly joint check-ins** (minimum) to discuss architectural decisions, data model alignment, and boundary cases (e.g., "Is 'What courses do I still need for my major?' an M1 query or an M2 query?").
- **Define a boundary document** — a short written agreement (1–2 pages) specifying which queries belong to M1, which belong to M2, and how ambiguous queries are handled. Include this in both teams' `docs/` directories.
- **Share a common data directory structure** for academic data, so that the eventual integration doesn't require re-processing.

The goal is that M1 and M2 can be composed behind a single academic advising interface in the future with minimal rework.

---

9. Cross-Module Considerations

Even outside the M1/M2 pairing, teams should be aware of how their modules relate to others:

- **A5 (Rubric Engine) → A1, A2, A3:** The rubric format defined in this guide (Section 4) is the contract between A5's output and the other autograding modules' input. A5 should generate rubrics in this format; A1, A2, and A3 should be able to consume rubrics in this format.
- **A4 (Repository Analyzer)** will use other teams' repositories as test data (with permission). Write commit history and branching patterns you'd want analyzed.
- **MaristChat modules (M1–M4)** share the chat endpoint contract. The future unified platform will route queries to the appropriate module, so consistent request/response shapes matter.

---

10. Getting Help

- **Technical questions about this guide:** Raise them in class or during office hours. If the answer affects all teams, we'll update this document.
- **Architecture decisions you're unsure about:** Bring them to your weekly check-in with the faculty PIs. Better to ask early than to rebuild later.
- **Cross-team coordination:** If your module has a dependency on or overlap with another team's module, initiate the conversation directly. Don't wait for us to broker it.

This guide will be updated as needed throughout the semester. Check the version number and last-updated date at the top.